



DISIM
Dipartimento di Ingegneria
e Scienze dell'Informazione
e Matematica



Software Engineering for Autonomous Systems

Autonomous Rendezvous Navigation for Docking Operations of Orbital Spacecrafts

Students:

Igli Balla – igli.balla@student.univaq.it

Mahira Ibnath Joytu - mahiraibnath.joytu@student.univaq.it

Urina Lama - urina.lama@student.univaq.it

Accepted:

Prof. Davide Di Ruscio

February 2, 2026

Contents

Introduction.....	3
Project Description and Architecture	3
Managed Resources	4
Sensors and State Estimation.....	4
Effectors and Actuators.....	4
Managed Resources Integration	5
Adaptation Goals	6
Decision Function Development Approach	6
Phase Classification.....	7
Learned Safe Speed Constraint.....	7
Docking Readiness Evaluation	8
Planner Decision Functions	8
Translational Control Law	8
Learned Maximum Force Constraint.....	8
Capture Feasibility Decision.....	9
Abort Probability and Abort Decision	9
Rotational Control and Torque Limiting	9
MAPE-K (Autonomous Rendezvous & Docking).....	10
MAPE-K-Based Autonomic Manager Architecture.....	11
Autonomous Rendezvous and Docking Operational Flow	12
Self-Adaptation.....	13

Introduction

Orbital spacecraft engage in multiple different operations during their lifecycle. Often, small spacecrafts are launched into space in order to connect to existing ones, adding new hardware or, as in the case of the International Space Station, delivering astronauts and supplies. This task usually requires detailed calculations and preparations that may take months ahead. Right now, this is done manually, as autonomous systems in orbit are still a new field of study. In this work, we propose a simple autonomous system based on the MAPE-K architecture, which allows smaller spacecraft to approach to the target, to simplify rendezvous and enable docking with less preparation needed.

Project Description and Architecture

This project was built using Basilisk, an Astrodynamics Simulation Framework, which is a framework that allows for creating immersive simulations of spacecraft using Python scripting. To develop this system, the simulator was used as the entry point to the MAPE-K architecture, and as the output, practically covering the Monitor and Executor, with slight additions. The simulator offers powerful tools for monitoring the navigation of the spacecraft initialized and provides real-time options for manipulating the state of these spacecrafts. The same team has also provided a Unity-based 3D visualization tool that displays the simulation in real time and offers a lot of features for more complex scenarios.

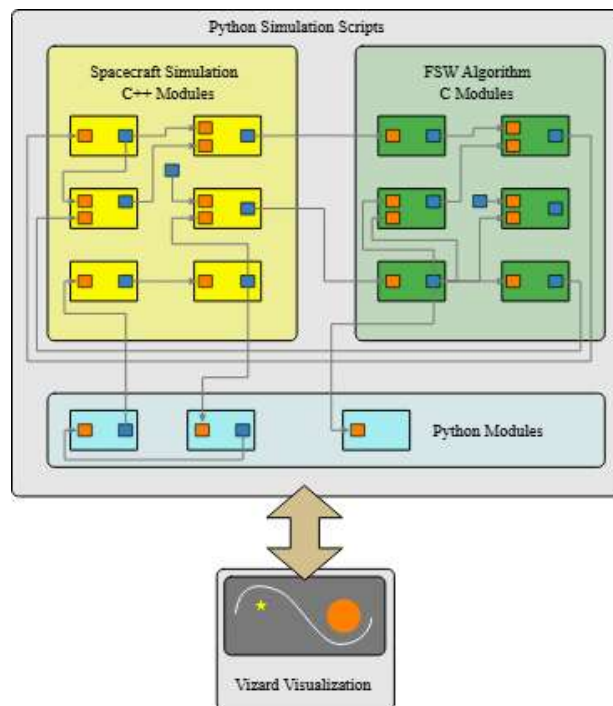


Figure 1. Basilisks Architecture.

The problem is designed as a problem of chaser and target. A small spacecraft is set in a random proximity towards a target, larger spacecraft. Its task is to approach it and create suitable conditions for docking. This system, depending on the distance between the spacecraft, calculated by reading the absolute positions of both spacecraft, must understand the current conditions, and perform the correct task to make sure the chaser correctly and safely approaches the target.

Managed Resources

The managed resources of this system are relative translational motion, relative attitude alignment, and actuation authority (force and torque budgets). Each resource is monitored, analyzed, and controlled independently, while being coordinated through a shared knowledge base that captures operational experience and safety constraints.

Translational motion is treated as a managed resource governing the relative position and velocity between the chaser and the target spacecraft in the Hill (LVLH) frame. Attitude alignment is managed as a separate resource, ensuring proper orientation of the chaser relative to the target's docking interface. Actuation authority is managed implicitly through force and torque saturation limits that evolve across mission phases and are refined using learned constraints from prior execution history.

Sensors and State Estimation

Relative translational state information is obtained through simulated navigation sensors implemented using Basilisk's *SimpleNav* module. These sensors provide estimates of each spacecraft's inertial position and velocity, which are transformed into relative Hill-frame quantities (relative position ρ and relative velocity $\dot{\rho}$). These measurements form the basis for monitoring range, closing speed, safety margins, and phase transitions within the Analyzer.

Attitude sensing is likewise provided through simulated navigation outputs, including Modified Rodrigues Parameters (MRPs) and angular rates for both spacecraft. The relative attitude between the chaser and the target is computed from these measurements and used to assess docking readiness and attitude safety. Together, the translational and attitude sensors provide a complete state estimate sufficient for autonomous proximity operations without requiring external supervision.

Effectors and Actuators

Translational control is achieved using a simulated external force effector (*ExtForceTorque*) attached to the chaser spacecraft. This effector applies commanded forces in the spacecraft body frame, enabling closed-loop control of relative motion. The Planner

computes these forces based on Hill-frame dynamics, proportional-derivative feedback, feedforward compensation, and learned constraints derived from the knowledge base. Force authority is explicitly limited as a managed resource, with tighter bounds enforced during close-range and docking phases to ensure safety and precision.

Attitude control is performed through a commanded external torque, also applied via the force-torque effector. The torque commands regulate the chaser's orientation relative to the target docking axis using proportional-derivative feedback on relative attitude and angular rate errors. Torque authority is similarly managed through saturation limits and may be further constrained using learned bounds stored in the knowledge base, allowing the system to adapt to observed stability and capture outcomes.

Managed Resources Integration

Each managed resource is evaluated independently in the Analyzer but coordinated in the Planner to produce consistent control actions. For example, translational capture feasibility influences whether aggressive closing maneuvers are permitted, while attitude readiness gates docking execution. The Knowledge Base records state, action, and outcome data across both translational and rotational domains, enabling learned safety envelopes, abort probabilities, and actuation limits to influence future planning decisions.

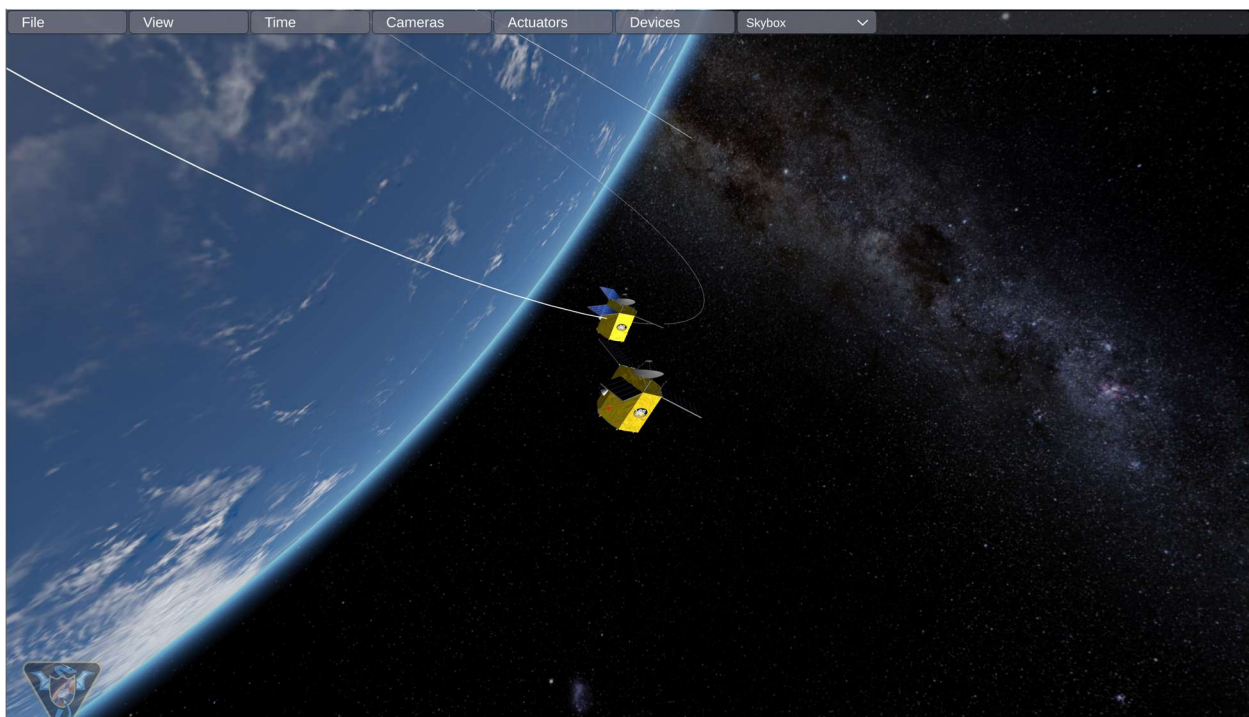


Figure 2. Screenshot of the implemented systems simulation

This architecture allows the autonomous system to progressively refine its use of sensors and effectors while preserving strict safety guarantees, demonstrating a realistic approach to resource-aware autonomy for spacecraft rendezvous and docking.

Adaptation Goals

The primary adaptation goal of the autonomic manager is to ensure safe, stable, and successful proximity operations and docking under varying dynamic conditions and uncertainty in relative motion. Safety is treated as the dominant objective, with the manager continuously adapting control authority and operational behavior to prevent excessive closing speeds, actuator saturation, or unstable capture dynamics. This is achieved by maintaining the system state within learned safety envelopes that evolve across mission phases.

A secondary adaptation goal is mission effectiveness, defined as the ability to progress from far-range rendezvous to stable capture and docking without unnecessary aborts or oscillatory behavior. The autonomic manager seeks to balance conservatism and performance by allowing more aggressive control when historical data indicates low risk, while tightening constraints when past executions reveal instability or failure modes. This enables the system to improve convergence and docking reliability over repeated runs.

A further goal is robustness to modeling error and unanticipated dynamics. Rather than relying exclusively on fixed analytical limits, the manager adapts its behavior based on observed outcomes, allowing it to compensate for unmodeled effects, imperfect gain tuning, and nonlinear interactions between translational and rotational dynamics. This makes the system resilient to variations in initial conditions and operating regimes.

Decision Function Development Approach

The decision functions are developed using a hybrid model-based and data-driven approach. Analytical physics-based models provide baseline constraints, such as range-dependent speed limits and Hill-frame feedforward dynamics, ensuring physically meaningful behavior even in the absence of prior experience. On top of these baseline models, empirical data collected during execution is stored in the knowledge base and used to refine decision boundaries based on observed success, instability, or abort events.

The autonomic manager applies these learned decision functions by modulating control authority and phase-dependent behavior rather than directly commanding discrete actions. For example, learned actuation limits cap commanded forces and torques, capture feasibility estimates influence whether closing maneuvers are permitted, and

abort probabilities gate transitions into high-risk phases. This indirect, constraint-based decision strategy preserves controller stability while enabling adaptive behavior.

Phase Classification

The operational phase is determined solely by the relative range magnitude:

$$\text{phase}(r) = \begin{cases} \text{FAR}, & r > 50 \\ \text{MID}, & 10 < r \leq 50 \\ \text{CLOSE}, & 1 < r \leq 10 \\ \text{DOCKING}, & r \leq 1 \end{cases}$$

This phase classification governs the selection of gains, constraints, and adaptation policies applied downstream.

Learned Safe Speed Constraint

A physics-based upper bound on closing speed is first defined as

$$v_{\text{phys}}(r) = \alpha \sqrt{\max(r, r_{\min})}$$

where α is a tuning constant (defaulting at 0.1) and $r_{\min}$ prevents singular behavior at short range. A learned safe speed bound $v_{\text{learned}}(r)$ is obtained from the knowledge base using historical safe executions. The effective safe speed is then

$$v_{\text{safe}}(r) = \min(v_{\text{phys}}(r), v_{\text{learned}}(r))$$

The instantaneous risk metric is defined as

$$\text{risk} = \frac{\|\dot{\mathbf{p}}\|}{v_{\text{safe}}(r)}$$

and the system is considered safe if

$$\text{risk} < \varepsilon_{\text{risk}}$$

where $\varepsilon_{\text{risk}}$ is arbitrary selected as 0.1.

Docking Readiness Evaluation

Docking readiness is determined using simultaneous positional, velocity, attitude, and angular-rate constraints:

$$\begin{aligned}\|\boldsymbol{\rho} \cdot \hat{\mathbf{n}}\| &< d_{\max} \\ \|\dot{\boldsymbol{\rho}}\| &< v_{\max} \\ \|\boldsymbol{\sigma}_{BT}\| &< \sigma_{\max} \\ \|\boldsymbol{\omega}_{BT}\| &< \omega_{\max}\end{aligned}$$

Docking readiness is declared only if all constraints are satisfied.

Planner Decision Functions

Translational Control Law

The nominal translational control command is defined as a Hill-frame PD controller with feedforward compensation:

$$\mathbf{a}_{\text{cmd}} = \mathbf{a}_{\text{ff}} - K_p \boldsymbol{\rho} - K_d (\dot{\boldsymbol{\rho}} - \dot{\boldsymbol{\rho}}_{\text{des}})$$

where the desired velocity is:

$$\dot{\boldsymbol{\rho}}_{\text{des}} = -v_{\max} \frac{\boldsymbol{\rho}}{\|\boldsymbol{\rho}\|}$$

The feedforward acceleration accounts for linearized orbital dynamics:

$$\mathbf{a}_{\text{ff}} = \begin{bmatrix} 3n^2 x + 2n\dot{y} \\ -2n\dot{x} \\ -n^2 z \end{bmatrix}$$

The commanded force is then

$$\mathbf{F}_H = m \mathbf{a}_{\text{cmd}}$$

Learned Maximum Force Constraint

A phase-dependent analytical force limit F_{phase} is combined with a learned maximum allowable force $F_{\text{learned}}(r)$:

$$F_{\max}(r) = \min(F_{\text{phase}}, F_{\text{learned}}(r))$$

The final force command is saturated as

$$\mathbf{F}_H \leftarrow \begin{cases} \mathbf{F}_H, & \|\mathbf{F}_H\| \leq F_{\max} \\ F_{\max} \frac{\mathbf{F}_H}{\|\mathbf{F}_H\|}, & \text{otherwise} \end{cases}$$

Capture Feasibility Decision

Capture feasibility is evaluated using a learned capture envelope:

$$\text{capture_possible} = \mathbb{I}(\|\mathbf{p} \cdot \hat{\mathbf{n}}\| \leq d_{\text{cap}} \wedge \|\dot{\mathbf{p}}\| \leq v_{\text{cap}}(d))$$

If capture is deemed infeasible in the CLOSE or DOCKING phases, the planner switches to a damping or hold strategy.

Abort Probability and Abort Decision

The abort probability is modeled as a learned function of relative distance and speed:

$$P_{\text{abort}} = f_{\text{abort}}(r, \|\dot{\mathbf{p}}\|)$$

An abort is triggered if

$$P_{\text{abort}} > P_{\text{thresh}} \text{ or } \text{risk} \geq 0.1$$

Upon abort, all commanded forces are set to zero and the system transitions to a safe-hold state.

Rotational Control and Torque Limiting

The rotational control law is defined as

$$\boldsymbol{\tau} = -K_r \boldsymbol{\sigma}_{BT} - K_\omega \boldsymbol{\omega}_{BT}$$

A learned torque limit $\tau_{\text{learned}}(\sigma)$ is combined with a fixed actuator limit $\tau_{\max}$:

$$\tau_{\max} = \min(\tau_{\max}, \tau_{\text{learned}}(\sigma))$$

Torque commands are saturated analogously to translational forces.

MAPE-K (Autonomous Rendezvous & Docking)

MAPE-K Element	Purpose in System	Core Responsibilities	Key Inputs	Key Outputs
Managed System	Physical system being controlled	Basilisk simulation of chaser–target rendezvous, including spacecraft dynamics, sensors, and actuators	Chaser & target spacecraft states	Updated spacecraft motion and attitude
Monitor (M)	Observe system state	Acquire navigation data and compute relative translational and rotational states	Basilisk navigation outputs (position, velocity, attitude, rates)	Relative state (ρ , $\dot{\rho}$, σ_{BT} , ω_{BT}), range, closing speed
Analyze (A)	Interpret current situation	Classify operational phase, assess safety/risk, evaluate docking readiness, detect abort conditions	Relative state from Monitor, learned limits from Knowledge	Phase label, risk/safety flags, dock_ready, capture_possible, abort
Plan (P)	Decide control actions	Select control mode (nominal, damping, hold, abort), compute force and torque commands under constraints	Analysis results, phase rules, knowledge-based envelopes	Saturated force command $\mathbf{F}$ and torque command $\boldsymbol{\tau}$
Execute (E)	Apply decisions	Send force and torque commands to the spacecraft effectors	Planned force and torque	Actuation applied to chaser spacecraft
Knowledge (K)	Store and refine experience	Log state–action–outcome data and maintain learned safety envelopes and limits	Runtime histories from all MAPE stages	Learned bounds (safe speed, max force/torque, capture envelopes, abort likelihood)

MAPE-K-Based Autonomic Manager Architecture

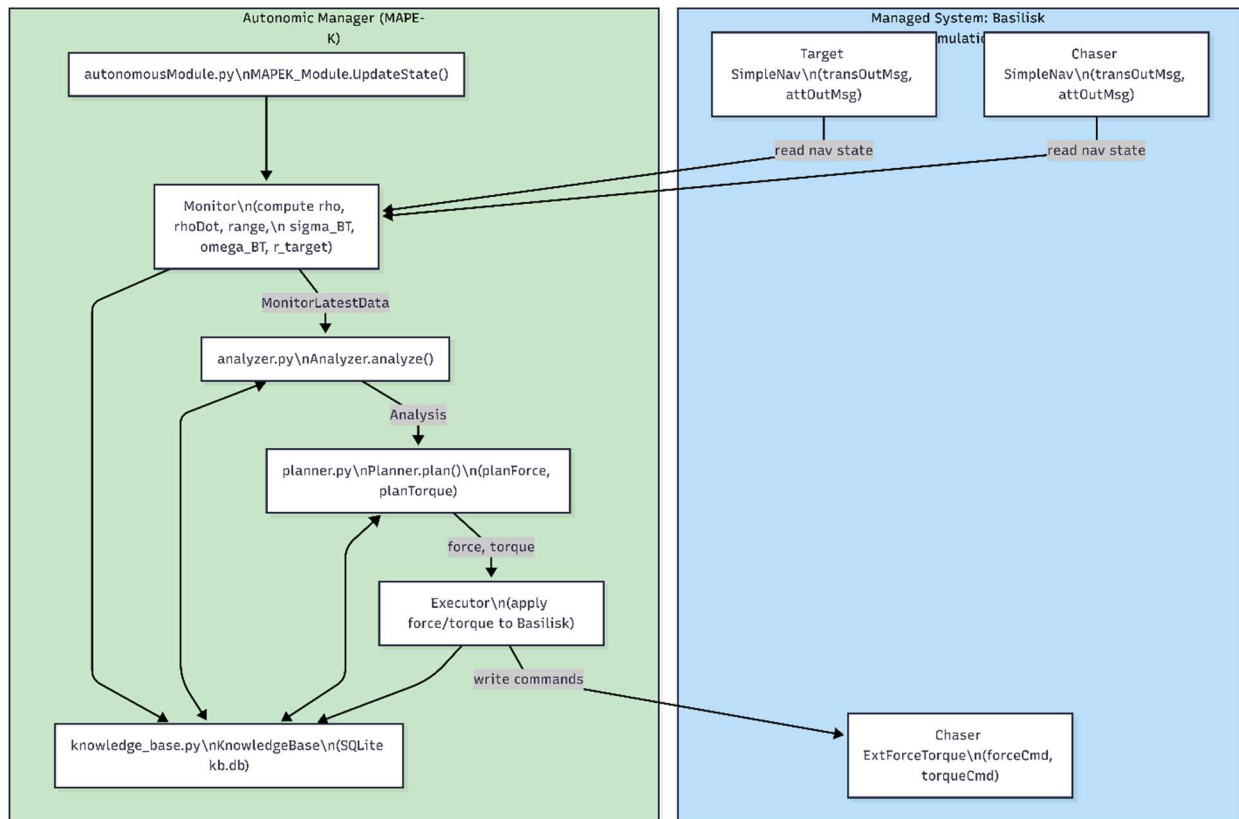


Figure 3. Component Diagram

This diagram illustrates the high-level software architecture of the autonomous rendezvous system. It shows the separation between the managed system (Basilisk simulation of the chaser and target spacecraft) and the autonomic manager implementing the MAPE-K loop. The diagram highlights how monitoring data flows from the simulation into the Monitor and Analyzer components, how planning decisions are produced using knowledge-based constraints, and how force and torque commands are executed back into the simulation.

Autonomous Rendezvous and Docking Operational Flow

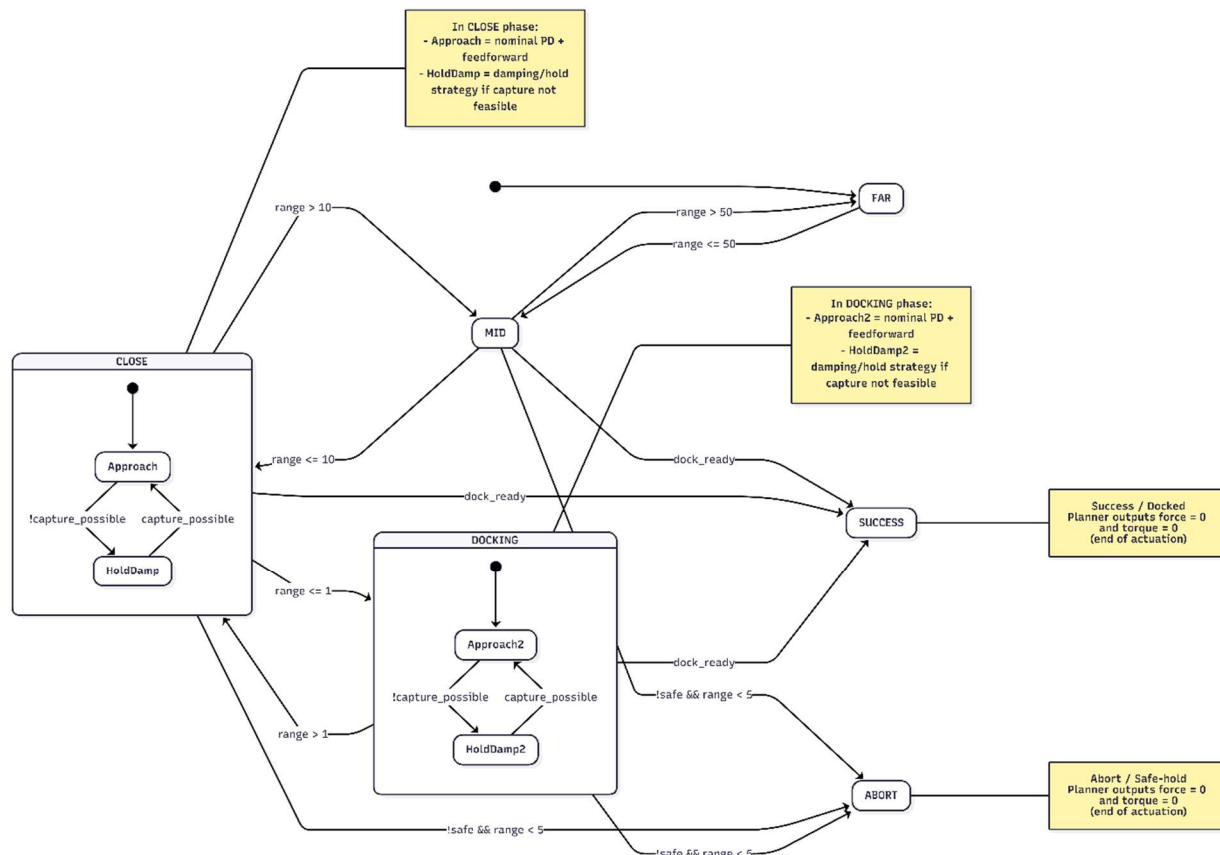


Figure 4. State diagram

This diagram represents the operational behavior of the autonomous system across different rendezvous phases. It captures the state transitions driven by relative distance, safety assessments, and docking readiness, including nominal approach, holding, successful docking, and abort conditions. The diagram emphasizes how safety and feasibility checks govern progression toward docking and trigger safe fallback behaviors when required.

Self-Adaptation

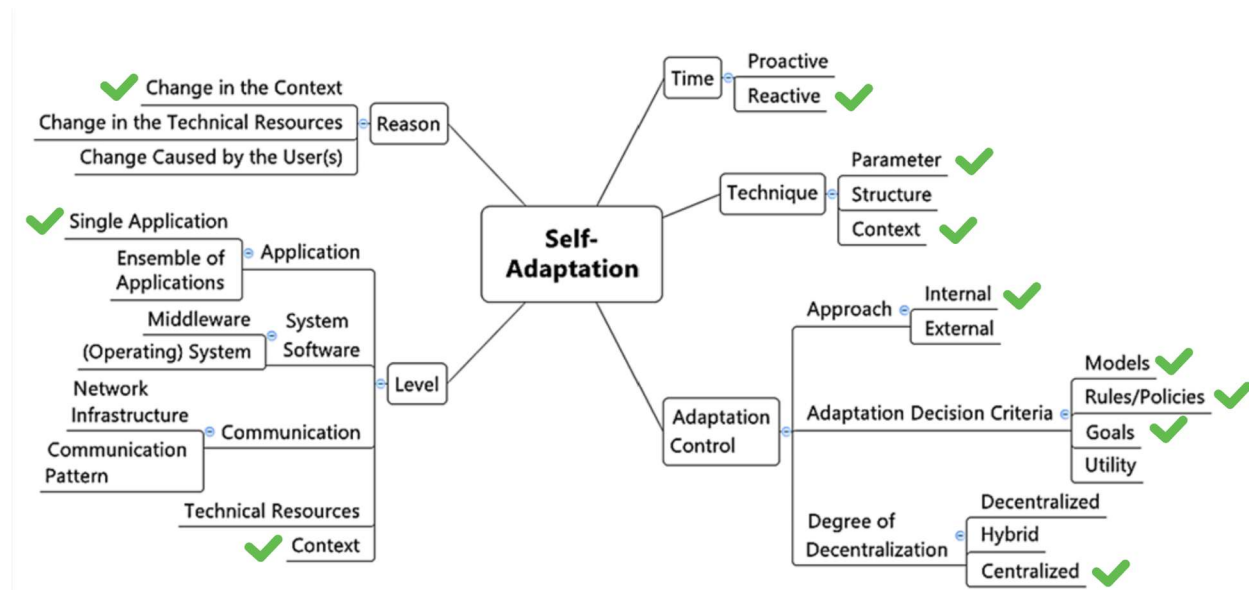


Figure 5. Self-Adaptation Chart

1. Time

- Reactive – It uses the current monitored state of the system to act on changing relative attitude and angular rate, and derived quantities such as range, closing speed, and risk metrics. The system has a knowledge base, but the learning is incremental, and not anticipatory.

2. Reason

- Change in the Context – The system adapts in response to changes in the operational context, as inferred from sensed relative motion, attitude, and orbital parameters.

3. Level

- Application – The system adapts in response to changes in the operational context, as inferred from sensed relative motion, attitude, and orbital parameters.

4. Technique

- Parameter – The adaptation relies on modifying the amount and direction of force and torque applied to the spacecraft.
- Context – The system operates at the application level, where mission-specific navigation and control decisions are made. These decisions directly influence low-level actuation, requiring minimal abstraction and low-latency execution, which is critical for close-proximity operations.

5. Adaptation Control

- Approach: Internal – An internal approach is employed, since a physics-based model is used. It is integrated into the simulator, where it is tightly coupled to the resources and executes as part of the same control loop.
- Adaptation Decision Criteria
 - i. Models: A physics-based model (Hill–Clohessy–Wiltshire dynamics) is used, which uses classical mechanics laws of motion to decide the force and torque needed to achieve the goals of the system.
 - ii. Rules/Policies: Depending on the range between spacecraft, the level of force that can be applied is decided, where FAR interactions are given more freedom, while CLOSE interactions are highly constrained to avoid potential collision risk. Only in the latter can torque be manipulated, to allow for the docking operation.
 - iii. Goals: The main objectives are to get the chasing spacecraft as close as possible to the target and initiate the docking procedure as safely as possible.

6. Degree of Decentralization

- Centralized: As such system would be employed into a spacecraft (which tend to have limited hardware capabilities), this system has to perform in a centralized manner.